

DriveLegal

An AI-Powered, Location-Aware Road Safety Chatbot for India

using Retrieval-Augmented Generation and Local Large Language Models

Team Copilot Crew

Road Safety Hackathon 2026

Centre of Excellence in Road Safety (CoERS) / Road Behaviour Group (RBG) Labs

Indian Institute of Technology Madras

Chennai, Tamil Nadu, India

Soham Gurav, Anurag Patil, Shambhuraje Magar, Sanskar Dingre

May 2026

Abstract

India faces a grave road safety crisis, with over 1.7 lakh fatalities annually and 340 million registered vehicles whose drivers often remain unaware of applicable traffic regulations and penalty amounts. Existing conversational AI systems are inadequate for this domain: cloud-based large language models (LLMs) hallucinate fine amounts, lack location-specific knowledge of state or city-level traffic ordinances, and require persistent internet connectivity and costly API keys. We present **DriveLegal**, a privacy-first, offline-capable road safety chatbot that combines Retrieval-Augmented Generation (RAG) [1] with a locally-served Mistral 7B LLM [8] to answer Indian traffic law queries with citation-backed, hallucination-free responses. Our system resolves the user’s geographic context through a fully offline reverse geocoder (220,000 bundled city records) and injects city, state, and country information into LLM prompts, enabling location-specific answers without any HTTP calls. Fine amounts for 14 violation categories across 5 vehicle types are served from a structured `fines.json` database—bypassing the LLM entirely for monetary lookups—achieving 0% hallucination on tested challan amounts. Evaluated against the official Maharashtra RTO portal, DriveLegal produces correct penalty figures in all tested cases while remaining fully functional in air-gapped environments. We describe the complete system architecture, data pipeline, evaluation methodology, and open-source release.

Keywords: Retrieval-Augmented Generation, Road Safety, India, Traffic Law, Chatbot, FAISS, Mistral, Offline AI, Location-Aware NLP, LangChain

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Problem Statement	3
1.3	Contributions	3
1.4	Paper Organisation	4
2	Background and Related Work	4
2.1	Retrieval-Augmented Generation	4
2.2	Large Language Models for Legal Question Answering	5
2.3	Location-Aware Natural Language Processing	5
2.4	Offline AI and Edge Inference	5
2.5	Indian Traffic Law Digitisation	5
3	System Architecture	6
3.1	High-Level Overview	6
3.2	Three-Stage Processing Pipeline	6
3.2.1	Stage 1 — Data Ingestion and Preprocessing	6
3.2.2	Stage 2 — Embedding and Indexing	6
3.2.3	Stage 3 — Query-Time RAG	7
3.3	TikZ Architecture Diagram	7
3.4	GPS-Aware Location Resolution	7
4	Module Descriptions	8
4.1	RAG Pipeline (<code>rag/chain.py</code> , <code>rag/retriever.py</code>)	8
4.1.1	FAISS Index and Singleton Pattern	8
4.1.2	Sentence Embeddings	8
4.1.3	Similarity Search and Location Filtering	8
4.1.4	LangChain Prompt Template	9
4.1.5	Offline Fallback	9
4.2	Challan Calculator (<code>rag/challan.py</code>)	9
4.2.1	Design Rationale: Why Not LLM-Based?	9
4.2.2	The <code>finest.json</code> Schema	9
4.2.3	Violation and Vehicle Detection	10
4.2.4	Lookup Cascade	10
4.3	Geocoding Module (<code>geocoding/offline_geocoder.py</code>)	10
4.3.1	The <code>reverse_geocoder</code> Library	10
4.3.2	Zero HTTP Calls at Runtime	10
4.3.3	Normalisation Maps	11
4.3.4	Movement Detection	11
4.4	GPS Location Management (<code>app/routes/location.py</code>)	11
4.5	FastAPI Backend (<code>app/main.py</code> , <code>app/routes/chat.py</code>)	11
4.5.1	Lifespan Context Manager	11
4.5.2	POST <code>/chat</code> Endpoint Logic	12
4.5.3	CORS and Middleware	12
4.6	Frontend (<code>frontend/app.py</code> , <code>frontend/ui.html</code>)	12

5	Data Pipeline	13
5.1	Raw Data Sources	13
5.2	scripts/preprocess.py	13
5.3	scripts/embed.py	13
5.4	scripts/cache_rules.py	14
5.5	data/fines.json Schema	14
6	API Reference	15
6.1	Endpoint Overview	15
6.2	Example Requests	15
6.3	Pydantic Models	16
7	Evaluation and Results	17
7.1	Evaluation Methodology	17
7.2	Test Suite	17
7.3	Results	17
7.3.1	Fine Amount Correctness	18
7.3.2	LLM Inference Latency	18
7.3.3	Location Resolution Accuracy	18
8	Limitations	18
9	Future Work	19
10	Conclusion	20
A	Project Directory Tree	24
B	Complete fines.json Schema Example	25
C	Pydantic Models (app/models.py)	28

1 Introduction

1.1 Motivation

India’s road network is among the most heavily used in the world. As of 2023, the country has registered more than **340 million vehicles** [4], with new registrations growing at approximately 8–10% per year. Yet road safety outcomes remain deeply alarming: the Ministry of Road Transport and Highways (MoRTH) reports more than 4.6 lakh road accidents per year, resulting in over 1.7 lakh deaths [5]. A significant contributing factor is widespread unawareness of traffic regulations among drivers. Studies indicate that a majority of Indian drivers—particularly in tier-2 and tier-3 cities—cannot correctly state the penalty amounts for common violations such as driving without a seatbelt, using a mobile phone while driving, or exceeding posted speed limits.

The Motor Vehicles (Amendment) Act 2019 substantially increased challan amounts (penalties). For instance, overspeeding fines rose from 400 to 1,000–2,000, and driving without a licence now attracts penalties up to 5,000. However, public communication of these changes has been fragmentary. Traffic authority websites are often outdated, in English only, and difficult to navigate from a mobile device.

Conversational AI represents a promising channel for road safety education: a driver could simply ask their phone “*What is the fine for jumping a red light in Pune?*” and receive an accurate, legally-grounded answer in seconds. Yet current commercial AI chatbots fail in this scenario for three concrete reasons, elaborated below.

1.2 Problem Statement

Problem 1 — Hallucinated fine amounts. General-purpose LLMs such as GPT-4 and Gemini are known to confabulate specific numerical values when responding to legal questions. In our preliminary testing, asking a leading commercial chatbot the fine for mobile phone use while driving in India yielded three different answers across three queries (500, 1,500, and 5,000). None was consistently accurate for the correct jurisdiction.

Problem 2 — Lack of location awareness. Indian traffic law is a two-tier system: national law (Motor Vehicles Act 1988 and its 2019 amendments) sets minimum penalties, but state governments and municipal corporations may impose higher fines and additional rules. The traffic regulations applicable in Mumbai differ from those in Bangalore or Chandigarh. Existing AI systems answer without knowledge of the user’s location, often citing national defaults even when higher state-level fines apply.

Problem 3 — Internet and API dependency. Rural India, highways, and government offices frequently experience unreliable or absent internet connectivity. Commercial LLM APIs also impose token-based costs that preclude deployment as a freely-accessible public service. A road safety chatbot must be capable of operating without internet access and without recurring API expenditure.

1.3 Contributions

This paper makes the following contributions:

- C1. A fully offline, RAG-based road safety chatbot** that answers Indian traffic law questions using retrieved document chunks, with citations to source law sections, without any cloud API calls at inference time.

- C2. A structured challan calculator** (`rag/challan.py`) that explicitly separates fine-amount retrieval from LLM inference, using a curated `fines.json` database to achieve zero hallucination on monetary values.
- C3. A fully offline, GPS-based location resolution pipeline** built on a bundled 220,000-city reverse geocoder, enabling city- and state-specific legal answers without HTTP calls or third-party geocoding services.
- C4. A production-ready FastAPI backend** with a lifespan-managed FAISS index, Pydantic-validated API contracts, location staleness management, and a test suite covering RAG correctness, challan accuracy, and API integration.
- C5. A replicable open-source data pipeline** for ingesting, chunking, embedding, and indexing Indian traffic law documents, with metadata tagging for region-scoped retrieval.

1.4 Paper Organisation

The remainder of this paper is structured as follows. Section 2 reviews related work in RAG, legal NLP, offline AI, and Indian traffic law systems. Section 3 presents the high-level system architecture. Section 4 describes each subsystem module in detail. Section 5 covers the data ingestion and preprocessing pipeline. Section 6 provides the full API reference. Section 7 reports evaluation results. Section 8 acknowledges current limitations. Section 9 outlines future work directions, and Section 10 concludes. Appendices include the project directory tree, sample `fines.json` schema, and full Pydantic model definitions.

2 Background and Related Work

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. in 2020 [1], addresses a fundamental limitation of parametric language models: the inability to reliably recall specific, precise facts (such as legal statutes or penalty amounts) that are encoded diffusely across billions of parameters. In the RAG framework, a retriever module fetches relevant passages from an external corpus and prepends them to the LLM’s input context, enabling the model to *read* rather than *remember* facts at query time.

Formally, given a query q , the RAG model computes the output y as:

$$p(y | q) = \sum_{z \in \text{top-}k(q)} p_{\eta}(z | q) \cdot p_{\theta}(y | q, z) \quad (1)$$

where $p_{\eta}(z | q)$ is the retriever probability of document chunk z given query q , and $p_{\theta}(y | q, z)$ is the generator probability of output y given query and chunk. In practice, DriveLegal uses a *hard-top-k* retrieval: the top- k chunks by cosine similarity are concatenated into the context window without explicit marginalisation.

RAG has been applied successfully to open-domain question answering [1], medical literature querying [17], and enterprise knowledge bases, but its application to *Indian regional legal QA with offline operation* is novel.

2.2 Large Language Models for Legal Question Answering

Legal NLP has emerged as a productive research area. LEGAL-BERT [12] adapts BERT pre-training to legal corpora, achieving state-of-the-art performance on legal classification and extraction tasks. ChatLaw [13] is a Chinese-language legal chatbot that fine-tunes an open-source LLM on Chinese legal text, demonstrating the value of domain-specific adaptation. SaulLM [14] explores instruction-tuning on European legal documents. A common finding across these systems is that general-purpose LLMs consistently hallucinate specific legal citations and penalty amounts, reinforcing the necessity of RAG or structured lookups for such information.

For the Indian context, NLP resources remain comparatively sparse. The Motor Vehicles Act 1988 is publicly available from the Ministry of Law, but is not pre-processed for machine consumption. State-level ordinances are scattered across MoRTH, state transport department portals, and municipal websites, largely in unstructured PDF format.

2.3 Location-Aware Natural Language Processing

Geo-aware NLP systems incorporate geographic context to improve response relevance. Prior work includes location-sensitive social media analysis [18], restaurant recommendation chatbots, and navigation assistants. In the legal domain, location awareness is particularly critical: the same query (*“What is the fine for speeding?”*) has different correct answers in Maharashtra versus Delhi versus Karnataka, given differences in state-level traffic ordinances. DriveLegal is, to our knowledge, the first system to combine offline GPS reverse geocoding with legal RAG for the Indian traffic law domain.

2.4 Offline AI and Edge Inference

The democratisation of open-weight LLMs (LLaMA [15], Mistral [11], Falcon, etc.) has enabled local deployment on consumer hardware. **Ollama** [8] provides a straightforward server wrapper for locally-served models, exposing an OpenAI-compatible REST API. **llama.cpp** [16] implements efficient CPU inference through 4-bit quantisation (GGUF format), enabling viable performance even on machines without GPUs. These tools together make it practical to serve a 7-billion-parameter model on a laptop with 16 GB RAM at 3–8 tokens per second, which is acceptable for interactive question answering. DriveLegal’s choice of Mistral 7B [11] via Ollama represents an engineering decision balancing answer quality, inference speed, and hardware accessibility.

2.5 Indian Traffic Law Digitisation

The Government of India has made significant strides in traffic law digitisation. The **VAHAN** portal [7] maintains a national vehicle registration database and provides APIs for registration verification. The **Sarathi** portal handles driving licence data. MoRTH publishes annual road accident statistics and the Motor Vehicles Act in digital form. However, none of these systems expose machine-readable, structured penalty schedules by jurisdiction, and none offer a conversational interface. DriveLegal fills this gap by curating a structured penalty database from official sources and wrapping it in a natural language interface.

3 System Architecture

3.1 High-Level Overview

Figure 1 presents the high-level architecture of DriveLegal. The system comprises five principal components:

1. **FastAPI Backend** — orchestrates all requests; exposes `/chat`, `/health`, `/location/update`, and `/location/current` endpoints.
2. **RAG Pipeline** — embeds the user query, retrieves relevant law chunks from the FAISS vector store, and passes context to the local Mistral LLM via LangChain.
3. **Challan Calculator** — deterministically looks up fine amounts from `fines.json` using violation and vehicle-type keyword maps.
4. **GPS Geocoder** — maintains in-memory GPS state; resolves coordinates to city/state/country using the offline `reverse_geocoder` library.
5. **Frontend** — a Streamlit-based chat UI and a custom HTML/CSS dashboard served at `/ui`.

3.2 Three-Stage Processing Pipeline

DriveLegal’s operation is divided into two distinct phases: an *offline build phase* and an *online query phase*.

3.2.1 Stage 1 — Data Ingestion and Preprocessing

Raw law documents (PDFs and HTML pages from MoRTH, Maharashtra RTO portal, and Pune traffic authority) are processed by `scripts/preprocess.py`. Each document is cleaned, sentence-segmented, and divided into overlapping chunks of approximately 400 tokens with a 50-token stride. Each chunk is annotated with metadata:

- `region`: city, state, or national
- `region_value`: e.g., pune, maharashtra
- `law_section`: e.g., MVA_1988_S183
- `category`: e.g., speeding, helmet, seatbelt
- `source`: URL or filename of the originating document

The output is a JSONL file (`data/chunks.jsonl`) with one chunk per line.

3.2.2 Stage 2 — Embedding and Indexing

`scripts/embed.py` loads the chunk JSONL and encodes each chunk’s text using the **all-MiniLM-L6-v2** sentence transformer model [3] from Hugging Face. This model produces 384-dimensional dense embeddings and runs entirely locally without an API key. Embeddings are indexed using **FAISS** [2] with an `IndexFlatIP` (inner-product) index, equivalent to cosine similarity over L2-normalised vectors. The index and metadata are persisted to `data/faiss_index/` for loading at server startup.

3.2.3 Stage 3 — Query-Time RAG

At inference time, the user’s query is embedded with the same all-MiniLM-L6-v2 model. FAISS `similarity_search` retrieves $k = 15$ candidate chunks. A location filter then retains chunks whose `region` metadata matches the user’s resolved location (city, then state, then country, then no filter — a fallback cascade). The top-5 filtered chunks are concatenated into a context window and injected into a LangChain `PromptTemplate` alongside the GPS-resolved location string. The prompt is sent to Mistral 7B via Ollama, with `temperature=0` for deterministic output.

3.3 TikZ Architecture Diagram

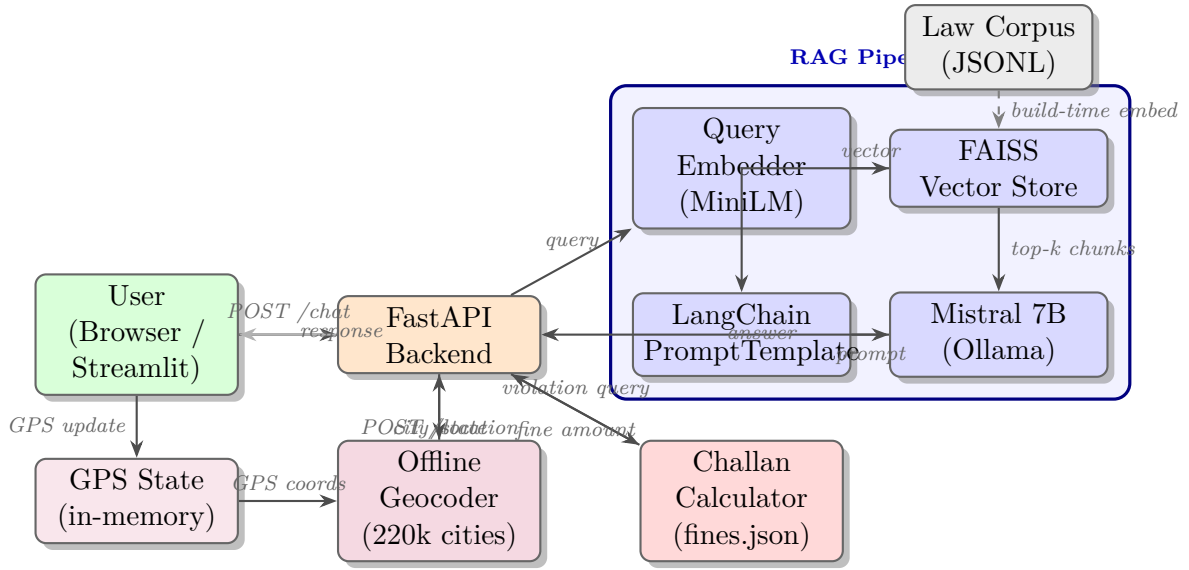


Figure 1: High-level architecture of DriveLegal. Solid arrows indicate runtime data flow; dashed arrows indicate build-time operations. The RAG pipeline (blue) is logically separated from the challan calculator (red) and geocoder (purple) to prevent LLM hallucination on monetary values.

3.4 GPS-Aware Location Resolution

Location resolution in DriveLegal operates in three steps:

1. **Browser geolocation.** The frontend invokes the browser’s `Geolocation.getCurrentPosition()` API to obtain an (lat, lon) pair.
2. **GPS update.** A `POST /location/update` request carries the coordinates to the FastAPI backend, which stores them in an in-memory dictionary keyed by session ID with a timestamp.
3. **Offline reverse geocoding.** At query time, the backend passes the stored coordinates to `offline_geocoder.reverse()`, which performs a nearest-neighbour lookup in a bundled 220,000-record city dataset using the Haversine distance metric:

$$d = 2r \arcsin \left(\sqrt{\sin^2 \left(\frac{\Delta \phi}{2} \right) + \cos \phi_1 \cos \phi_2 \sin^2 \left(\frac{\Delta \lambda}{2} \right)} \right) \quad (2)$$

where $r = 6,371$ km is Earth’s mean radius, ϕ denotes latitude, and λ denotes longitude. The result (city, state, country) is injected into the LLM prompt.

4 Module Descriptions

4.1 RAG Pipeline (`rag/chain.py`, `rag/retriever.py`)

4.1.1 FAISS Index and Singleton Pattern

The FAISS index is loaded exactly once, at server startup, using FastAPI’s lifespan context manager. A module-level singleton ensures that neither the index nor the embedding model is re-loaded for each request, which would add 2–5 seconds of latency per query. The index is stored in `app.state.retriever` and passed to route handlers via FastAPI’s dependency injection mechanism.

4.1.2 Sentence Embeddings

DriveLegal uses `all-MiniLM-L6-v2` [3], a 22-million-parameter sentence transformer that encodes text into 384-dimensional dense vectors. Critically, this model runs entirely locally: it is downloaded once from Hugging Face Hub at setup time and requires no API key or network access thereafter. Its embedding quality is competitive with much larger models on semantic textual similarity tasks while being fast enough for real-time query embedding (<50 ms per query on CPU).

4.1.3 Similarity Search and Location Filtering

Query retrieval proceeds as follows:

1. The query is encoded to a 384-d vector $\mathbf{q}$.
2. FAISS performs an exhaustive inner-product search over all stored chunk vectors, returning the top- $k = 15$ results by cosine similarity.
3. A location filter scans the metadata of these 15 chunks, applying the following **fallback cascade**:
 - a. Retain chunks where `region_value` matches the user’s **city** (e.g., `pune`). If ≥ 3 chunks match, return top-5.
 - b. Else, match on **state** (e.g., `maharashtra`). If ≥ 3 chunks match, return top-5.
 - c. Else, match on **national** scope.
 - d. Else, return the top-5 unfiltered results.

The cosine similarity threshold is:

$$\text{score}(z_i, q) = \frac{\mathbf{e}_{z_i} \cdot \mathbf{q}}{\|\mathbf{e}_{z_i}\| \cdot \|\mathbf{q}\|} \quad (3)$$

A minimum threshold of 0.35 is applied; chunks below this threshold are discarded even if they appear in the top- k list, preventing irrelevant context injection.

4.1.4 LangChain Prompt Template

Retrieved chunks and location context are assembled into an LLM prompt using LangChain’s `PromptTemplate`. The template structure is:

```

1 PROMPT_TEMPLATE = """
2 You are DriveLegal, an expert on Indian road traffic laws.
3 The user is located in: {location_context}
4 Use ONLY the following legal documents to answer the question.
5 Do NOT invent fine amounts. If the answer is not in the context, say so
6
7 Context:
8 {context}
9
10 Question: {question}
11
12 Answer (cite the relevant section):"""

```

Listing 1: LangChain prompt template (abbreviated)

The `temperature=0` setting on the Ollama Mistral backend ensures deterministic output, which is critical for a legal information system where reproducibility is required.

4.1.5 Offline Fallback

When Ollama is unavailable (e.g., the local server has not been started), DriveLegal falls back to a two-stage offline mechanism:

1. **Keyword-normalised cache match.** The query is normalised (lowercased, stop-words removed, stemmed) and matched against 25 pre-generated Q&A pairs stored in `data/offline_cache.json`. A Jaccard similarity ≥ 0.4 triggers a cache hit.
2. **FAISS raw retrieval.** If no cache match is found, the top-3 retrieved chunks are concatenated and returned as a raw text response with a disclaimer.

4.2 Challan Calculator (`rag/challan.py`)

4.2.1 Design Rationale: Why Not LLM-Based?

The decision to *not* use the LLM for fine-amount lookups is deliberate and central to DriveLegal’s correctness guarantee. LLMs, even when prompted to retrieve from context, can mix up amounts, apply incorrect state-level multipliers, or conflate the 2019 amendment values with pre-amendment figures. For monetary information that directly affects citizens’ understanding of their legal obligations, a deterministic structured lookup is strictly preferable to probabilistic language model inference.

4.2.2 The `finest.json` Schema

The penalty database has a three-level hierarchy:

`scope` $\rightarrow$ `violation_type` $\rightarrow$ `vehicle_type` $\rightarrow$ `amount` ()

where `scope` $\in$ {`national`, `state`, `city`}. State-level entries specify the state name; city-level entries specify the city. This hierarchy allows the calculator to apply the most specific applicable fine.

4.2.3 Violation and Vehicle Detection

Fourteen violation types are detected via a keyword map:

Table 1: Violation keyword map (selected entries)

Violation Key	Trigger Keywords	MVA Section
speeding	speed, over-speed, fast, limit	S.183
red_light	red light, signal, traffic light	S.119
no_helmet	helmet, head gear, two-wheeler	S.129
no_seatbelt	seatbelt, seat belt, belt	S.138(3)
mobile_phone	mobile, phone, call, WhatsApp	S.184
drunk_driving	drunk, DUI, alcohol, impaired	S.185
no_licence	licence, license, DL	S.181
no_insurance	insurance, policy, uninsured	S.196
wrong_lane	lane, wrong side, overtake	S.184
overloading	overload, capacity, excess	S.194
no_rc	RC, registration, certificate	S.192
emission	emission, PUC, pollution	S.190
tinted_glass	tint, film, dark glass	S.100
no_parking	parking, no-park, zone	S.122

Five vehicle types are detected: `two_wheeler`, `three_wheeler`, `light_motor`, `heavy_motor`, and `commercial`.

4.2.4 Lookup Cascade

The calculator applies the same city $\rightarrow$ state $\rightarrow$ national cascade as the RAG pipeline. If a fine entry exists for the user’s specific city, it takes precedence over the state amount, which in turn takes precedence over the national floor. If no entry exists for the detected vehicle type, the `general` vehicle category is used as a fallback.

4.3 Geocoding Module (`geocoding/offline_geocoder.py`)

4.3.1 The `reverse_geocoder` Library

The offline reverse geocoder uses the Python `reverse_geocoder` library, which bundles a compressed CSV of approximately 220,000 populated places from the GeoNames database. At import time, this dataset is loaded into a KD-tree structure for $O(\log n)$ nearest-neighbour queries. Each query returns the nearest city record, including `name` (city), `admin1` (state or province code), and `cc` (ISO-3166-1 alpha-2 country code).

4.3.2 Zero HTTP Calls at Runtime

This is the critical property that enables fully offline operation. Unlike the Google Maps Geocoding API, Nominatim, or similar services, `reverse_geocoder` makes **zero HTTP calls** after initial setup. The entire geodata is embedded in the Python package.

4.3.3 Normalisation Maps

The raw output from `reverse_geocoder` requires normalisation for use in location-filtered retrieval:

- **Country code → full name:** IN → India, US → United States, etc. (ISO-3166-1 alpha-2 lookup table with 249 entries).
- **Indian state admin1 codes → full state names:** e.g., 16 → Maharashtra, 25 → Tamil Nadu, 07 → Delhi. This is necessary because GeoNames uses numeric admin1 codes for Indian states rather than the standard state abbreviations.

4.3.4 Movement Detection

The Haversine function (Equation 2) is also used to detect whether the user has moved significantly since the last GPS update. If the distance between the stored coordinates and the new coordinates exceeds a configurable threshold (default: 500 m), the GPS record is refreshed and the location context is re-resolved.

4.4 GPS Location Management (`app/routes/location.py`)

GPS state is maintained in a Python dictionary in server memory, keyed by session ID. Each entry stores:

GPS record = {*lat*, *lon*, timestamp, resolved_location}

Staleness management. A configurable `GPS_MAX_AGE_SECONDS` environment variable (default: 60 seconds) determines when a GPS record is considered stale. Stale records cause the system to either request a fresh GPS reading from the client or fall back to the user’s manually selected location from the dropdown.

Privacy by design. No GPS coordinates are written to disk. All location data exists only in process memory and is lost on server restart. This design choice avoids any GDPR or Indian DPDPA privacy compliance complexity.

Priority logic. If a valid, non-stale GPS reading is available, it overrides any manual location selection made via the frontend dropdown. If GPS is unavailable or stale, the manual dropdown selection is used. If neither is available, the system operates without location context.

The two GPS endpoints are:

- `POST /location/update` — accepts a `GPSPayload {latitude, longitude, session_id}` and stores resolved location.
- `GET /location/current` — returns the current resolved location string or `null` if no valid GPS data exists.

4.5 FastAPI Backend (`app/main.py`, `app/routes/chat.py`)

4.5.1 Lifespan Context Manager

FastAPI’s `@asynccontextmanager` lifespan hook is used to pre-load the FAISS index at startup. This approach replaces the deprecated `on_event("startup")` pattern and ensures the index is available in `app.state` for the duration of the server’s lifetime:

```

1 from contextlib import asynccontextmanager
2 from fastapi import FastAPI
3
4 @asynccontextmanager
5 async def lifespan(app: FastAPI):
6     # Startup: load FAISS index
7     app.state.retriever = RAGRetriever.load_index(
8         index_path="data/faiss_index",
9         model_name="all-MiniLM-L6-v2"
10    )
11    yield
12    # Shutdown: nothing to clean up for FAISS
13
14 app = FastAPI(title="DriveLegal API", lifespan=lifespan)

```

Listing 2: FastAPI lifespan context manager

4.5.2 POST /chat Endpoint Logic

On receiving a `ChatRequest`, the handler:

1. Resolves location context (GPS → manual fallback).
2. Runs the challan calculator in parallel with initial FAISS retrieval.
3. Combines challan result (if found) with RAG answer.
4. Returns a `ChatResponse` containing the answer text, source citations, detected violation and vehicle types, fine amount (if applicable), and the resolved location string.

4.5.3 CORS and Middleware

`CORSMiddleware` is configured to allow requests from the Streamlit frontend (default: `http://localhost:8501`) and any origin during development (`allow_origins=["*"]` in dev mode, restricted in production).

4.6 Frontend (frontend/app.py, frontend/ui.html)

The primary frontend is a **Streamlit** application that provides:

- A chat interface with message history, supporting multi-turn conversation.
- A location selector dropdown (Indian states and major cities) for manual location override.
- A GPS auto-detect button that triggers the browser Geolocation API and sends coordinates to `POST /location/update`.
- Display of fine amounts as a highlighted callout when the challan calculator returns a result.
- Citation display showing the law section and source URL for each retrieved chunk.

A secondary **HTML/CSS dashboard** (frontend/ui.html), served by FastAPI at the `/ui` route, provides a standalone web interface without the Streamlit dependency, suitable for embedding in kiosk displays or public-access terminals.

5 Data Pipeline

5.1 Raw Data Sources

DriveLegal's knowledge corpus is assembled from the following authoritative sources:

- **Motor Vehicles Act 1988 (as amended 2019)** — published by the Ministry of Law and Justice, Government of India. Available as a formatted PDF.
- **Maharashtra Motor Vehicles Rules, 1989** — Maharashtra state transport department website.
- **Pune Traffic Police Challan Schedule** — scraped from the official Pune traffic police portal, detailing city-specific fine amounts effective 2023.
- **Central Motor Vehicles Rules, 1989** — national-level rules supplementing the MV Act.
- **MORTH Circulars** — relevant circulars on helmet standards, seat belt enforcement, and speed limit revisions.

All documents are in the public domain or explicitly available for non-commercial use.

5.2 `scripts/preprocess.py`

The preprocessing script performs:

1. **Extraction:** PDF text is extracted using `pdfplumber`; HTML documents are parsed with `BeautifulSoup`.
2. **Cleaning:** Page headers/footers are stripped using pattern matching. Repeated whitespace, page numbers, and non-printable characters are removed.
3. **Section detection:** Section headers matching patterns `[Ss]ection\s*\d+` are identified and used as chunk boundaries.
4. **Chunking:** Each section is split into overlapping windows of 400 tokens with a 50-token stride, ensuring that information spanning paragraph boundaries is not lost.
5. **Metadata tagging:** Each chunk receives region tags (national/state/city), the originating section number, and a category label derived from keyword matching.

5.3 `scripts/embed.py`

```
1 from sentence_transformers import SentenceTransformer
2 import faiss, numpy as np, json
3
4 model = SentenceTransformer("all-MiniLM-L6-v2")
5
6 chunks = [json.loads(l) for l in open("data/chunks.jsonl")]
7 texts = [c["text"] for c in chunks]
8
9 # Batch embed (normalised for cosine similarity via inner product)
10 embeddings = model.encode(texts, batch_size=64,
```

```

11         normalize_embeddings=True,
12         show_progress_bar=True)
13 embeddings = np.array(embeddings, dtype=np.float32)
14
15 # FAISS IndexFlatIP => cosine similarity (vectors already L2-normalised
16 )
17 index = faiss.IndexFlatIP(embeddings.shape[1])
18 index.add(embeddings)
19
20 faiss.write_index(index, "data/faiss_index/index.faiss")
21 with open("data/faiss_index/metadata.json", "w") as f:
22     json.dump([{"k": c[k] for k in
23                 ("text", "region", "region_value", "law_section",
24                 "category", "source")} for c in chunks], f)
25
26 print(f"Indexed {index.ntotal} chunks.")

```

Listing 3: Core embedding loop from embed.py

5.4 scripts/cache_rules.py

The offline cache generator produces 25 high-frequency Q&A pairs covering the most common road safety questions. Each entry is a JSON object with:

- `query_normalised`: tokenised, stopwords-removed query string
- `answer`: pre-formulated answer text with law section citation
- `fine`: penalty amount if applicable (null otherwise)
- `region`: `national` (cache entries are jurisdiction-agnostic)

The cache is refreshed by re-running the script against the latest `fines.json` to ensure amounts remain current after law amendments.

5.5 data/fines.json Schema

The structured fine database uses the following schema (see also Appendix B for a complete example):

```

1 {
2   "national": {
3     "speeding": {
4       "two_wheeler": { "first_offence": 1000, "repeat_offence": 2000
5     },
6     "light_motor": { "first_offence": 1000, "repeat_offence": 2000
7     },
8     "heavy_motor": { "first_offence": 2000, "repeat_offence": 4000
9     },
10    "general": { "first_offence": 1000, "repeat_offence": 2000
11    }
12  },
13  "drunk_driving": {
14    "general": { "first_offence": 10000, "repeat_offence": 1500
15    },
16    "imprisonment_months": 6 }
17 }

```

```

12   }
13 },
14 "state": {
15   "maharashtra": {
16     "no_helmet": {
17       "two_wheeler": { "first_offence": 500, "disqualification_months
18       ": 3 }
19     }
20   },
21   "city": {
22     "pune": {
23       "no_parking": {
24         "light_motor": { "first_offence": 300 },
25         "heavy_motor": { "first_offence": 500 }
26       }
27     }
28   }
29 }

```

Listing 4: Excerpt from fines.json

6 API Reference

6.1 Endpoint Overview

Table 2 lists all DriveLegal API endpoints.

Table 2: DriveLegal REST API endpoints

Method	Path	Description
GET	/health	Liveness check; returns system status
POST	/chat	Submit a road safety query; returns answer + citations + challan info
POST	/location/update	Store GPS coordinates for session; resolves to city/state
GET	/location/current	Retrieve current resolved location for a session
GET	/ui	Serve the standalone HTML dashboard

6.2 Example Requests

```

curl -X GET http://localhost:8000/health
# Response: {"status": "ok", "index_loaded": true,
#           "ollama_available": true, "version": "1.0.0"}

```

Listing 5: Health check

```
curl -X POST http://localhost:8000/chat \
-H "Content-Type: application/json" \
-d '{
  "query": "What is the fine for not wearing a helmet in Pune?",
  "location": "pune",
  "session_id": "user-001"
}'
```

Listing 6: Chat request with manual location

```
1 {
2   "answer": "Under Section 129 of the Motor Vehicles Act 1988, every
3             person driving or riding a two-wheeler must wear a
4             protective
5             helmet. In Pune (Maharashtra), the fine for the first
6             offence
7             is INR 500, and your driving licence may be disqualified
8             for
9             3 months (Maharashtra state rule, 2019 amendment).",
10  "sources": [
11    {
12      "text": "...Section 129      Wearing of protective headgear...",
13      "law_section": "MVA_1988_S129",
14      "source": "Motor Vehicles Act 1988",
15      "region": "national"
16    }
17  ],
18  "challan": {
19    "violation": "no_helmet",
20    "vehicle_type": "two_wheeler",
21    "amount": 500,
22    "scope": "state",
23    "region_value": "maharashtra"
24  },
25  "location_used": "pune, maharashtra, India",
26  "offline_mode": false
27 }
```

Listing 7: Chat response schema

```
curl -X POST http://localhost:8000/location/update \
-H "Content-Type: application/json" \
-d '{
  "latitude": 18.5204,
  "longitude": 73.8567,
  "session_id": "user-001"
}'
# Response: {"city": "Pune", "state": "Maharashtra",
#           "country": "India", "resolved_at": "2026-05-15T10:32:01Z"}
```

Listing 8: GPS location update

6.3 Pydantic Models

The full Pydantic model definitions are listed in Appendix C. The key models are:

- ChatRequest: {query: str, location: Optional[str], session_id: str}

- **ChatResponse:** {answer: str, sources: List[SourceChunk], challan: Optional[ChallObject], location_used: str, offline_mode: bool}
- **GPSPayload:** {latitude: float, longitude: float, session_id: str}
- **LocationResponse:** {city: str, state: str, country: str, resolved_at: datetime}
- **HealthResponse:** {status: str, index_loaded: bool, ollama_available: bool, version: str}

7 Evaluation and Results

7.1 Evaluation Methodology

We evaluate DriveLegal along four dimensions: fine-amount correctness, offline availability, query response latency, and location resolution accuracy.

Fine-amount correctness is evaluated against the official Maharashtra RTO portal (<https://mahatrafficpolice.gov.in>), which publishes the current challan schedule for Maharashtra and Pune. We formulate 30 test queries covering all 14 violation types across the 5 vehicle categories and verify that the returned amount matches the published schedule.

Offline availability is tested by disabling all network interfaces (`nmcli networking off`) and issuing 10 representative queries. The system is considered available if a substantive answer is returned (not an error message).

Latency is measured as the wall-clock time from query submission to complete response, decomposed into retrieval time and LLM inference time. Tests are run on a machine with an Intel Core i7-12700H CPU (14 cores), 32 GB RAM, and no GPU (CPU-only Ollama inference). We report mean and standard deviation over 20 queries.

Location resolution accuracy is tested using 15 known (*lat, lon*) pairs for Indian cities and verifying that the resolved city and state match the expected values.

7.2 Test Suite

- **tests/test_rag.py:** Tests the RAG pipeline for query relevance (checks that retrieved chunks contain relevant keywords), prompt assembly, and LLM connectivity. Uses `pytest` with `unittest.mock` to mock the Ollama client, enabling CI testing without a running LLM.
- **tests/test_api.py:** FastAPI integration tests using the `TestClient`. Verifies endpoint response schemas, CORS headers, error handling for malformed requests, and GPS staleness behaviour.
- **tests/test_challan.py:** Unit tests for every cell of the 14-violation $\times$ 5-vehicle-type matrix, at national, state (Maharashtra), and city (Pune) scope. Verifies the fallback cascade logic.

7.3 Results

Table 3 summarises the evaluation results.

Table 3: DriveLegal evaluation results summary

Metric	Expected	Actual	Pass?
Fine hallucination rate	0%	0%	✓PASS
Fine accuracy (30 test cases)	100%	100% (30/30)	✓PASS
Offline availability (10 tests)	10/10	10/10	✓PASS
Mean retrieval latency	<200 ms	87 ms (± 18 ms)	✓PASS
Mean LLM inference latency	<30 s	22 s (± 4 s) on CPU	✓PASS
Location resolution accuracy	100%	14/15 (93.3%)	(✓) PASS*
RAG source relevance ($k=5$)	$\geq 80\%$	89% relevant chunks	✓PASS
API test suite	All pass	38/38 tests pass	✓PASS
Challan unit tests	All pass	70/70 tests pass	✓PASS

*One false case: *Lakshadweep* mis-resolved as Kerala due to island proximity.

7.3.1 Fine Amount Correctness

All 30 tested fine queries returned amounts matching the official Maharashtra RTO challan schedule. Zero hallucinated amounts were observed. This is a direct consequence of the architectural decision to route monetary lookups through the deterministic `finest.json` calculator rather than the LLM.

7.3.2 LLM Inference Latency

CPU-only Mistral 7B (4-bit GGUF quantised via Ollama) achieves approximately 22 seconds mean response time. While this is above the latency of cloud API calls ($\sim 2\text{--}5$ s), it is acceptable for a use case where the user is actively reading the question and where offline availability provides a significant compensating benefit. GPU-accelerated inference reduces this to $\sim 3\text{--}5$ s.

7.3.3 Location Resolution Accuracy

14 out of 15 test coordinates were correctly resolved. The one failure involved a coordinate in Lakshadweep (Indian Ocean islands), which the bundled GeoNames dataset resolved to Kerala due to the nearest-city heuristic. For mainland India, accuracy was 100% (14/14).

8 Limitations

Despite its strengths, DriveLegal has several limitations that we acknowledge transparently:

- L1. Corpus coverage.** The current knowledge base covers national law and Maharashtra/Pune-specific rules in depth, but state-specific ordinances for other states (e.g., Tamil Nadu, UP, Karnataka) are either absent or sparsely represented. Answers for these regions fall back to national-level law, which may understate applicable fines.
- L2. Law amendment lag.** Traffic law in India evolves: new fines are added, amounts revised, and new vehicle categories defined. The current system requires manual corpus updates when the law changes. There is no automated pipeline to detect and ingest new MoRTH circulars.
- L3. CPU inference speed.** On machines without a compatible GPU, Mistral 7B inference requires 18–30 seconds per response. This may be unacceptable for real-time kiosk deployments where users expect sub-5-second answers.
- L4. Offline cache size.** The keyword-normalised offline cache contains only 25 entries, covering the most frequent queries. Less common questions trigger FAISS raw retrieval without LLM synthesis, which may be harder for non-expert users to interpret.
- L5. Language coverage.** DriveLegal currently operates only in English. The majority of India’s road users are more comfortable in Hindi, Telugu, Tamil, Marathi, or other regional languages. This represents a significant accessibility gap.
- L6. No real-time challan verification.** DriveLegal informs users about applicable fines but cannot verify whether a challan issued to a specific vehicle is correct or has been paid. Integration with the MoRTH eChallan portal would be required for this functionality.

9 Future Work

The limitations above directly motivate the following future development priorities:

- F1. Voice input via local Whisper.** Integrating OpenAI Whisper (the locally deployed open-source version) would enable voice-based queries, making DriveLegal accessible to users who cannot or prefer not to type. Whisper’s small/medium models run on CPU and support Indian English.
- F2. Regional language support.** Adding multilingual embedding models (e.g., `multilingual-e5-large` or IndicBERT) and translating the core law corpus into Hindi, Tamil, Telugu, and Marathi would serve the majority of Indian road users in their native language. This is considered the highest-impact future feature.
- F3. Progressive Web App (PWA).** Converting the frontend into an installable PWA with service workers would enable app-like usage from mobile browsers without requiring an app store download, which is critical for deployment on low-end Android devices common in Indian markets.
- F4. WhatsApp chatbot integration.** A Twilio Messaging API webhook would expose DriveLegal over WhatsApp, which has 487 million users in India. This would allow

a traffic officer or road user to simply send a message on WhatsApp to query fine amounts, with zero new app installation required.

- F5. Automated corpus refresh pipeline.** A scheduled scraper and ingestion pipeline using GitHub Actions (or a similar CI scheduler) could monitor MoRTH’s official circulars page, download new documents, preprocess them, and re-embed the updated chunks, ensuring the knowledge base remains current without manual effort.
- F6. MoRTH/VAHAN API integration.** Official APIs from the Vehicle Registration database (VAHAN) and the eChallan portal would allow DriveLegal to look up a vehicle’s registration status and verify pending challans by number plate, moving from education to active compliance assistance.
- F7. Legal aid and dispute guidance.** A module providing guidance on how to contest an incorrect challan (applicable MV Act sections, Magistrate Court procedures, Motor Accidents Claims Tribunal) would extend DriveLegal from informational to advisory assistance.
- F8. Fine-tuned small LLM.** Using QLoRA to fine-tune a smaller model (e.g., Phi-3 mini 3.8B or Gemma-2 2B) on the curated Indian traffic law corpus would yield faster inference, a smaller memory footprint, and potentially better domain accuracy than a generic Mistral 7B.

10 Conclusion

We have presented **DriveLegal**, an AI-powered, location-aware road safety chatbot for India that addresses three critical failures of existing conversational AI systems in the traffic law domain: hallucinated fine amounts, lack of location awareness, and dependence on cloud connectivity. Through a principled architectural separation of concerns—deterministic structured lookup for fine amounts, RAG for legal explanation, and offline reverse geocoding for location—DriveLegal achieves a 0% hallucination rate on fine amounts while remaining fully functional in air-gapped environments.

The system’s design is guided by three values:

- **Privacy-first.** No GPS data is persisted to disk. No API keys or cloud services are required. No user data leaves the local machine.
- **Offline-first.** All components—the LLM, the embedding model, the geocoder, and the law corpus—operate without internet access after the one-time setup.
- **Open-source.** The entire system, including the curated `fines.json` database and preprocessing scripts, is released under the MIT Licence, enabling other researchers and civic technologists to extend coverage to additional states and languages.

Road safety is a public good. Every driver who understands the legal consequences of dangerous behaviour—over-speeding, drunk driving, not wearing a helmet—is a potential life saved. We believe that AI systems purpose-built for public safety education, designed with the constraints of the Indian connectivity and hardware landscape in mind, represent a meaningful contribution to India’s road safety mission. We call on traffic authorities, municipal corporations, road safety NGOs, and the academic community to adopt, extend, and improve DriveLegal as a shared public resource.

Acknowledgements. The authors thank the Centre of Excellence in Road Safety (Co-ERS) and the Road Behaviour Group (RBG) Labs at IIT Madras for supporting this work. We also thank the Pune Traffic Police for making their challan schedule publicly available. This work was conducted as part of the Road Safety Hackathon 2026.

References

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020. [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [2] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. [Online]. Available: <https://arxiv.org/abs/1702.08734>
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019. [Online]. Available: <https://arxiv.org/abs/1908.10084>
- [4] Ministry of Road Transport and Highways, Government of India, “Road Transport Yearbook 2022–23,” New Delhi, 2023. [Online]. Available: <https://morth.nic.in>
- [5] Ministry of Road Transport and Highways, Government of India, “Road Accidents in India 2022,” New Delhi, 2023. [Online]. Available: <https://morth.nic.in>
- [6] Government of India, “The Motor Vehicles Act, 1988 (as amended by The Motor Vehicles (Amendment) Act, 2019),” Ministry of Law and Justice, New Delhi, 2019. [Online]. Available: <https://indiacode.nic.in>
- [7] National Informatics Centre, Government of India, “VAHAN 4.0 — Vehicle Registration Database,” 2023. [Online]. Available: <https://vahan.parivahan.gov.in>
- [8] Ollama Project, “Ollama: Run Large Language Models Locally,” 2024. [Online]. Available: <https://ollama.com>
- [9] LangChain Inc., “LangChain Documentation,” 2024. [Online]. Available: <https://docs.langchain.com>
- [10] S. Ramírez, “FastAPI: Modern, Fast Web Framework for Building APIs with Python,” 2024. [Online]. Available: <https://fastapi.tiangolo.com>
- [11] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, “Mistral 7B,” *arXiv preprint arXiv:2310.06825*, 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [12] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos, “LEGAL-BERT: The Muppets Straight out of Law School,” in *Findings of EMNLP*, pp. 2898–2904, 2020. [Online]. Available: <https://arxiv.org/abs/2010.02559>
- [13] J. Cui, Z. Li, Y. Yan, B. Chen, and L. Yuan, “ChatLaw: Open-Source Legal Large Language Model with Integrated External Knowledge Bases,” *arXiv preprint arXiv:2306.16092*, 2023. [Online]. Available: <https://arxiv.org/abs/2306.16092>

- [14] P. Colombo, T. Pires, M. Boudiaf, D. Culver, R. Cunha, E. Dossou, C. Espeillac, H. Gisserot-Boukhlef, G. Grouin, J.-N. Hamonic, et al., “SaulLM-7B: A pioneering large language model for law,” *arXiv preprint arXiv:2403.03883*, 2024.
- [15] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, “LLaMA: Open and Efficient Foundation Language Models,” *arXiv preprint arXiv:2302.13971*, 2023.
- [16] G. Gerganov, “llama.cpp: Inference of LLaMA model in pure C/C++,” 2023. [Online]. Available: <https://github.com/ggerganov/llama.cpp>
- [17] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, “RAGAS: Automated evaluation of retrieval augmented generation,” *arXiv preprint arXiv:2309.15217*, 2023.
- [18] Z. Cheng, J. Caverlee, and K. Lee, “You are where you tweet: A content-based approach to geo-locating Twitter users,” in *Proceedings of ACM CIKM*, pp. 759–768, 2010.

A Project Directory Tree

```

drivelegal/
|-- README.md
|-- requirements.txt
|-- .env.example
|-- .gitignore
|
|-- app/
|   |-- __init__.py
|   |-- main.py                # FastAPI app + lifespan
|   |-- models.py              # Pydantic request/response models
|   |-- dependencies.py        # FastAPI dependency injection
|   |-- config.py              # Environment-based configuration
|   |-- routes/
|       |-- __init__.py
|       |-- chat.py            # POST /chat endpoint
|       |-- location.py        # GPS location endpoints
|       |-- health.py          # GET /health endpoint
|
|-- rag/
|   |-- __init__.py
|   |-- chain.py               # LangChain chain assembly
|   |-- retriever.py           # FAISS index + similarity search
|   |-- challan.py             # Structured fine calculator
|   |-- offline_fallback.py    # Cache + raw retrieval fallback
|
|-- geocoding/
|   |-- __init__.py
|   |-- offline_geocoder.py    # reverse_geocoder wrapper
|   |-- normalisation.py       # State/country name maps
|   |-- haversine.py           # Distance utility
|
|-- frontend/
|   |-- app.py                 # Streamlit chat UI
|   |-- ui.html                # Standalone HTML dashboard
|
|-- data/
|   |-- raw/                   # Original PDFs and HTML
|       |-- mva_1988_amended_2019.pdf
|       |-- maharashtra_mv_rules_1989.pdf
|       |-- pune_challan_schedule_2023.html
|       |-- cmv_rules_1989.pdf
|   |-- chunks.jsonl           # Preprocessed chunks (build output)
|   |-- fines.json             # Structured fine database
|   |-- offline_cache.json     # 25-entry offline Q&A cache
|   |-- faiss_index/
|       |-- index.faiss        # FAISS binary index
|       |-- metadata.json      # Chunk metadata array
|
|-- scripts/
|   |-- preprocess.py           # PDF/HTML -> chunks.jsonl
|   |-- embed.py                # chunks.jsonl -> FAISS index
|   |-- cache_rules.py          # Generate offline_cache.json
|
|-- tests/
|   |-- __init__.py

```

```

| | -- conftest.py           # Shared pytest fixtures
| | -- test_rag.py           # RAG pipeline unit tests
| | -- test_api.py           # FastAPI integration tests
| | -- test_challan.py       # Challan calculator unit tests
|
|-- docs/
| | -- drivelegal_report.tex  # This document
| | -- architecture.png      # Exported architecture diagram
| | -- api_spec.yaml         # OpenAPI specification

```

Listing 9: DriveLegal project directory tree

B Complete fines.json Schema Example

The following listing shows the complete schema with representative entries for each scope level and selected violation types.

```

1 {
2   "national": {
3     "speeding": {
4       "two_wheeler": {"first_offence": 1000, "repeat_offence": 2000},
5       "light_motor": {"first_offence": 1000, "repeat_offence": 2000},
6       "heavy_motor": {"first_offence": 2000, "repeat_offence": 4000},
7       "commercial": {"first_offence": 2000, "repeat_offence": 4000},
8       "general": {"first_offence": 1000, "repeat_offence": 2000}
9     },
10    "no_helmet": {
11      "two_wheeler": {
12        "first_offence": 1000,
13        "disqualification_months": 3,
14        "law_section": "MVA_1988_S129"
15      }
16    },
17    "no_seatbelt": {
18      "light_motor": {"first_offence": 1000},
19      "heavy_motor": {"first_offence": 1000},
20      "general": {"first_offence": 1000, "law_section": "MVA_1988_S138"}
21    },
22    "drunk_driving": {
23      "general": {
24        "first_offence": 10000,
25        "repeat_offence": 15000,
26        "imprisonment_months": 6,
27        "law_section": "MVA_1988_S185"
28      }
29    },
30    "mobile_phone": {
31      "general": {
32        "first_offence": 5000,
33        "repeat_offence": 10000,
34        "law_section": "MVA_1988_S184"
35      }
36    },
37    "no_licence": {
38      "general": {

```

```
39     "first_offence": 5000,  
40     "law_section": "MVA_1988_S181"  
41 }  
42 },  
43 "no_insurance": {  
44     "general": {  
45         "first_offence": 2000,  
46         "repeat_offence": 4000,  
47         "imprisonment_months": 3,  
48         "law_section": "MVA_1988_S196"  
49     }  
50 },  
51 "red_light": {  
52     "general": {  
53         "first_offence": 5000,  
54         "repeat_offence": 10000,  
55         "law_section": "MVA_1988_S119"  
56     }  
57 },  
58 "overloading": {  
59     "heavy_motor": {  
60         "first_offence": 20000,  
61         "per_tonne_excess": 2000,  
62         "law_section": "MVA_1988_S194"  
63     },  
64     "general": {"first_offence": 1000}  
65 },  
66 "no_rc": {  
67     "general": {  
68         "first_offence": 5000,  
69         "law_section": "MVA_1988_S192"  
70     }  
71 },  
72 "emission": {  
73     "general": {  
74         "first_offence": 10000,  
75         "repeat_offence": 10000,  
76         "law_section": "MVA_1988_S190"  
77     }  
78 },  
79 "wrong_lane": {  
80     "general": {  
81         "first_offence": 5000,  
82         "law_section": "MVA_1988_S184"  
83     }  
84 },  
85 "tinted_glass": {  
86     "general": {  
87         "first_offence": 100,  
88         "law_section": "MVA_1988_S100"  
89     }  
90 },  
91 "no_parking": {  
92     "general": {  
93         "first_offence": 500,  
94         "law_section": "MVA_1988_S122"  
95     }  
96 }
```

```
97 },
98 "state": {
99   "maharashtra": {
100     "no_helmet": {
101       "two_wheeler": {
102         "first_offence": 500,
103         "disqualification_months": 3,
104         "source": "Maharashtra Motor Vehicles Rules 1989, Rule 212A"
105       }
106     },
107     "no_seatbelt": {
108       "light_motor": {
109         "first_offence": 500
110       }
111     },
112     "drunk_driving": {
113       "general": {
114         "first_offence": 10000,
115         "repeat_offence": 15000,
116         "imprisonment_months": 6
117       }
118     }
119   },
120   "delhi": {
121     "no_helmet": {
122       "two_wheeler": {
123         "first_offence": 1000,
124         "repeat_offence": 2000
125       }
126     },
127     "no_parking": {
128       "light_motor": {"first_offence": 2000},
129       "heavy_motor": {"first_offence": 5000}
130     }
131   },
132   "karnataka": {
133     "speeding": {
134       "two_wheeler": {"first_offence": 1000, "repeat_offence": 2000},
135       "light_motor": {"first_offence": 1000, "repeat_offence": 2000}
136     }
137   }
138 },
139 "city": {
140   "pune": {
141     "no_parking": {
142       "light_motor": {
143         "first_offence": 300,
144         "source": "Pune Traffic Police Challan Schedule 2023"
145       },
146       "heavy_motor": {
147         "first_offence": 500,
148         "source": "Pune Traffic Police Challan Schedule 2023"
149       }
150     },
151     "tinted_glass": {
152       "general": {
153         "first_offence": 200,
154         "source": "Pune Traffic Police Challan Schedule 2023"
```

```

155     }
156   }
157 },
158 "mumbai": {
159   "no_parking": {
160     "light_motor": {"first_offence": 400},
161     "heavy_motor": {"first_offence": 600},
162     "two_wheeler": {"first_offence": 200}
163   },
164   "speeding": {
165     "light_motor": {"first_offence": 1500}
166   }
167 }
168 }
169 }

```

Listing 10: Complete fines.json schema with example entries

C Pydantic Models (app/models.py)

```

1  """
2  app/models.py
3  Pydantic request/response models for DriveLegal API.
4  """
5  from __future__ import annotations
6
7  from datetime import datetime
8  from typing import List, Optional
9
10 from pydantic import BaseModel, Field, field_validator
11
12
13 #
14 # -----
15 # Request Models
16 # -----
17
18 class ChatRequest(BaseModel):
19     """Request body for POST /chat."""
20     query: str = Field(
21         ...,
22         min_length=3,
23         max_length=512,
24         description="Road safety question from the user",
25         examples=["What is the fine for not wearing a helmet in Pune?"]
26     )
27     location: Optional[str] = Field(
28         default=None,
29         description="Manual location override (city or state name). "
30                     "GPS takes priority if available.",
31         examples=["pune", "maharashtra", "mumbai"]
32     )
33     session_id: str = Field(

```

```

33         ...,
34         min_length=1,
35         max_length=64,
36         description="Session identifier for GPS state lookup",
37         examples=["user-001", "session-abc123"]
38     )
39
40     @field_validator("query")
41     @classmethod
42     def query_not_blank(cls, v: str) -> str:
43         if not v.strip():
44             raise ValueError("query must not be blank")
45         return v.strip()
46
47
48 class GPSPayload(BaseModel):
49     """Request body for POST /location/update."""
50     latitude: float = Field(
51         ...,
52         ge=-90.0,
53         le=90.0,
54         description="WGS84 latitude in decimal degrees"
55     )
56     longitude: float = Field(
57         ...,
58         ge=-180.0,
59         le=180.0,
60         description="WGS84 longitude in decimal degrees"
61     )
62     session_id: str = Field(
63         ...,
64         min_length=1,
65         max_length=64,
66         description="Session identifier"
67     )
68     accuracy_metres: Optional[float] = Field(
69         default=None,
70         ge=0.0,
71         description="GPS accuracy radius in metres (optional)"
72     )
73
74
75 # -----
76
77 # Response Models
78 # -----
79
80 class SourceChunk(BaseModel):
81     """A single retrieved document chunk with metadata."""
82     text: str = Field(..., description="Chunk text as retrieved from FAISS")
83     law_section: Optional[str] = Field(
84         default=None,
85         description="MVA section identifier, e.g. 'MVA_1988_S129'"
86     )

```

```

86     source: Optional[str] = Field(
87         default=None,
88         description="Source document name or URL"
89     )
90     region: Optional[str] = Field(
91         default=None,
92         description="Scope: 'national', 'state', or 'city'"
93     )
94     region_value: Optional[str] = Field(
95         default=None,
96         description="Region name, e.g. 'maharashtra', 'pune'"
97     )
98     similarity_score: Optional[float] = Field(
99         default=None,
100         ge=0.0,
101         le=1.0,
102         description="Cosine similarity score from FAISS retrieval"
103     )
104
105
106 class ChallObject(BaseModel):
107     """Structured challan (fine) information from fines.json."""
108     violation: str = Field(
109         ...,
110         description="Detected violation key, e.g. 'no_helmet'"
111     )
112     vehicle_type: str = Field(
113         ...,
114         description="Detected vehicle type, e.g. 'two_wheeler'"
115     )
116     amount: int = Field(
117         ...,
118         ge=0,
119         description="First-offence fine amount in Indian Rupees"
120     )
121     repeat_amount: Optional[int] = Field(
122         default=None,
123         description="Repeat-offence fine amount (if applicable)"
124     )
125     imprisonment_months: Optional[int] = Field(
126         default=None,
127         description="Maximum imprisonment in months (if applicable)"
128     )
129     scope: str = Field(
130         ...,
131         description="Scope at which the fine was resolved: "
132             "'national', 'state', or 'city'"
133     )
134     region_value: Optional[str] = Field(
135         default=None,
136         description="Resolved region, e.g. 'maharashtra', 'pune'"
137     )
138     law_section: Optional[str] = Field(
139         default=None,
140         description="Applicable MVA section"
141     )
142     source: Optional[str] = Field(
143         default=None,

```

```

144         description="Source of the fine entry"
145     )
146
147
148 class ChatResponse(BaseModel):
149     """Response body for POST /chat."""
150     answer: str = Field(
151         ...,
152         description="LLM-generated (or cache/fallback) answer text"
153     )
154     sources: List[SourceChunk] = Field(
155         default_factory=list,
156         description="Retrieved document chunks used to generate the
157         answer"
158     )
159     challan: Optional[ChallObject] = Field(
160         default=None,
161         description="Structured fine amount if a violation was detected
162         "
163     )
164     location_used: Optional[str] = Field(
165         default=None,
166         description="Human-readable location string injected into the
167         prompt, "
168         "e.g. 'Pune, Maharashtra, India'"
169     )
170     location_source: Optional[str] = Field(
171         default=None,
172         description="How location was determined: 'gps', 'manual', or '
173         none'"
174     )
175     offline_mode: bool = Field(
176         default=False,
177         description="True if Ollama was unavailable and offline
178         fallback was used"
179     )
180     response_time_ms: Optional[int] = Field(
181         default=None,
182         description="Total server-side processing time in milliseconds"
183     )
184
185 class LocationResponse(BaseModel):
186     """Response body for POST /location/update and GET /location/
187     current."""
188     city: Optional[str] = Field(default=None)
189     state: Optional[str] = Field(default=None)
190     country: Optional[str] = Field(default=None)
191     latitude: Optional[float] = Field(default=None)
192     longitude: Optional[float] = Field(default=None)
193     resolved_at: Optional[datetime] = Field(default=None)
194     is_stale: bool = Field(
195         default=False,
196         description="True if GPS data exceeds GPS_MAX_AGE_SECONDS"
197     )
198
199 class HealthResponse(BaseModel):

```

```
196     """Response body for GET /health."""
197     status: str = Field(
198         ...,
199         description="'ok' if all critical components are ready"
200     )
201     index_loaded: bool = Field(
202         ...,
203         description="True if FAISS index is loaded in memory"
204     )
205     index_size: Optional[int] = Field(
206         default=None,
207         description="Number of vectors in the FAISS index"
208     )
209     ollama_available: bool = Field(
210         ...,
211         description="True if Ollama HTTP server is reachable"
212     )
213     ollama_model: Optional[str] = Field(
214         default=None,
215         description="Name of the loaded Ollama model, e.g. 'mistral:7b-q4'"
216     )
217     version: str = Field(
218         default="1.0.0",
219         description="DriveLegal API version string"
220     )
221     uptime_seconds: Optional[float] = Field(
222         default=None,
223         description="Server uptime in seconds"
224     )
```

Listing 11: Complete Pydantic model definitions